Bisectr

A cost-and-performance gate for container services, with automatic bisect as the safety net.

Bisectr runs your pull request and your `main` branch side by side on AWS Fargate under identical load, compares them statistically, and tells you on the PR whether the change made the service slower, flakier, or more expensive to run. If a regression slips through anyway, Bisectr binary-searches the commit range and names the commit responsible.

> Catch the regression before it merges. If it merges anyway, find the commit in minutes.

Project status

This README describes the target design. The code samples are reference sketches that have not been run end to end, and every number in the results tables is marked `TODO(measure)`. Replace those with your own measured values as you build. Do not publish unmeasured numbers.

Table of contents

1. The problem
2. What Bisectr does
3. How it works
4. Architecture
5. The probe task
6. Measurement and decision rule
7. Cost model
8. Tech stack
9. Repository layout
10. Getting started
11. Configuration reference
12. Security
13. Observability and cost control
14. Evaluation
15. Testing and CI
16. Roadmap
17. Related work
18. Limitations
19. License

---

1. The problem

Teams find out a service got slower or more expensive after it ships. Unit tests do not catch a new N+1 query, a heavier serialization path, or a dependency upgrade that doubles CPU per request. By the time a latency alarm or a surprise bill appears, many commits have merged and nobody knows which one is responsible.

Two questions come up again and again:

1. **Before merge:** does this pull request make the service slower or costlier to run?
2. **After merge:** which of these 20 commits caused the regression?

Bisectr answers both with one measurement engine.

2. What Bisectr does

Mode 1: Gate (prevent)

On every pull request, Bisectr builds the PR image, runs it and the current `main` baseline on Fargate at the same time under the same synthetic load, and posts a comment with:

- p95 latency and error rate, with a statistical verdict
- **CPU per request** and peak memory, measured from the running containers
- an estimated monthly cost change at your target traffic
- a pass, warn, or fail check on the pull request

Example comment (illustrative values, not real results):

Metric	`main` (baseline)	This PR	Change
p95 latency	41 ms	63 ms	+54%
Error rate	0.0%	0.0%	none
CPU per 1,000 requests	0.82 CPU-s	1.01 CPU-s	+23%
Peak memory	118 MB	121 MB	+3%

Verdict: **FAIL (latency)**. Mann-Whitney p < 0.001, p95 ratio 1.54, 3 of 3 replicas worse than baseline. Sizing estimate at 500 RPS: 2 tasks become 3 tasks, about +$XX per month (computed from your pricing config).

Mode 2: Bisect (cure)

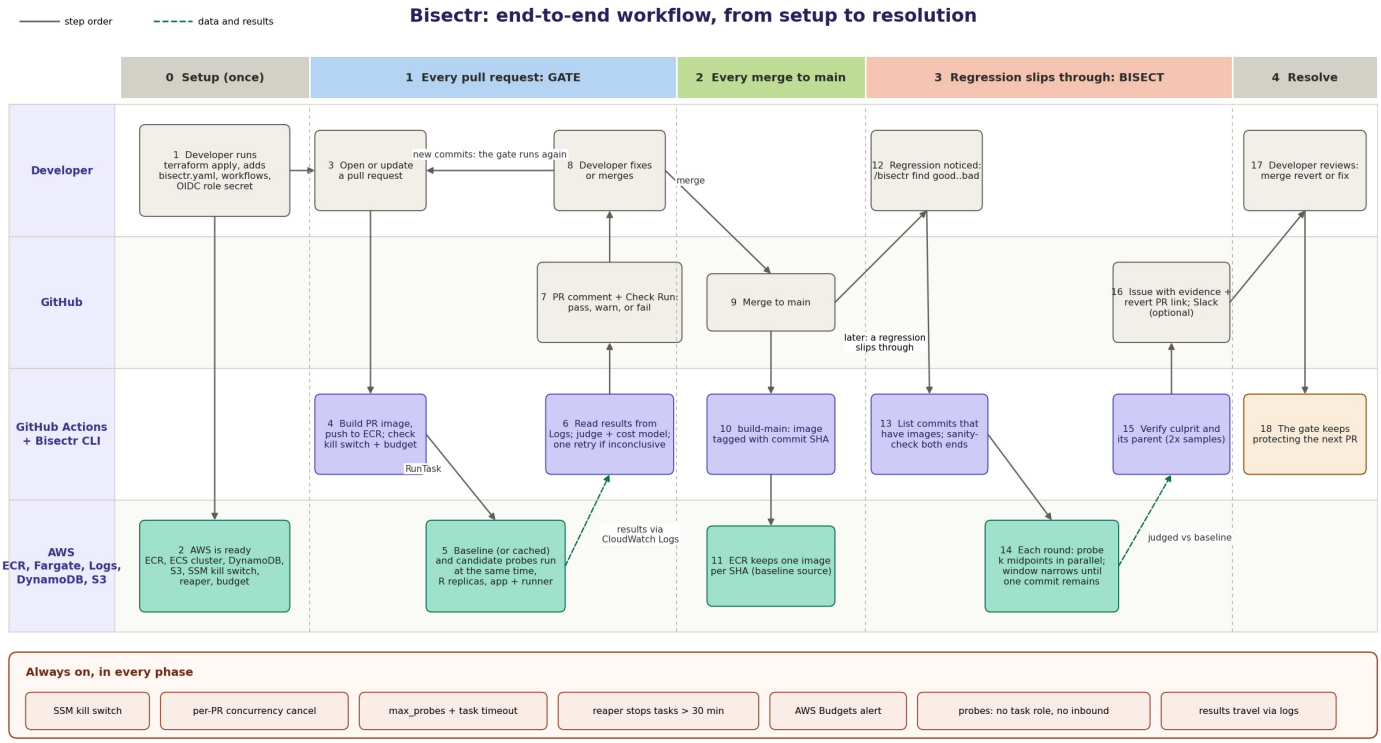
When a regression is found after merge, run `/bisectr find <good>..<bad>` on a pull request or issue, trigger the workflow manually, or wire a CloudWatch alarm to it. Bisectr tests candidate commits in parallel, narrows the range, verifies the answer, and opens an issue with the evidence.

Key features

- Replicated, simultaneous A/B probes on Fargate, so baseline and candidate share the same time window and noise
- An open-loop load generator that does not hide latency spikes (no coordinated omission)
- A decision rule that requires statistical significance, a material effect size, and agreement between replicas
- Measured CPU and memory per request, converted to a sizing and cost estimate
- Bisect with `skip`, retry, and a verification step that reports an ambiguous window instead of a confident wrong commit
- Runs as a GitHub Action with a small Terraform module. There is no always-on backend
- Security by design: probe tasks have no task role and no inbound network access

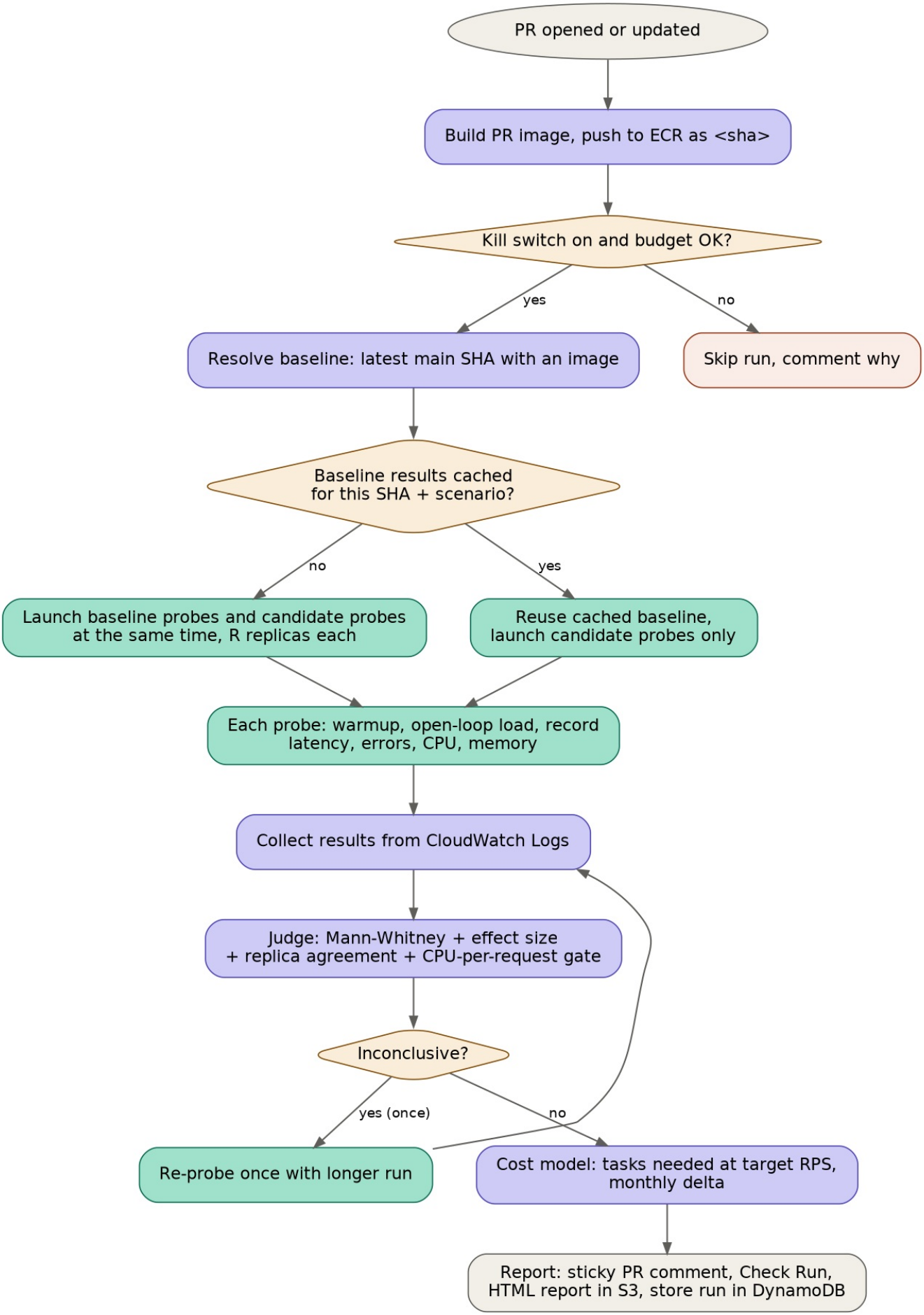
3. How it works

End-to-end view



The swimlane shows the whole life of the system: one-time setup, the gate on every pull request, the image build on every merge, the bisect when a regression slips through, and the resolution. The bottom band lists the guardrails that run in every phase. The two sections below zoom into the gate and the bisect.

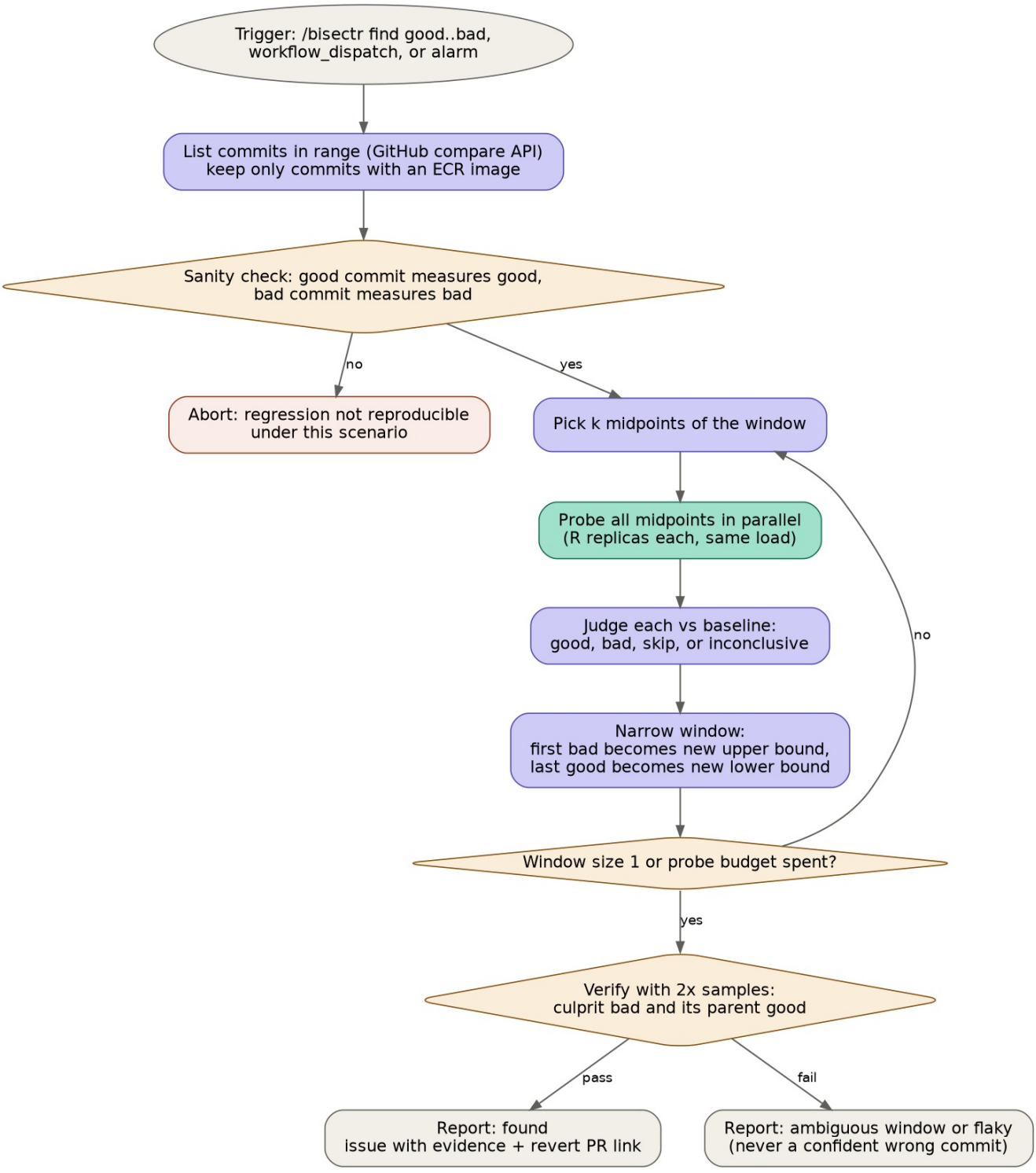
3.1 Gate workflow



1. **Trigger.** A pull request is opened or updated. A concurrency group cancels superseded runs to save money.

2. **Build.** The workflow builds the PR image and pushes it to ECR tagged with the commit SHA.
3. **Guardrails.** The CLI checks the kill switch (an SSM parameter) and the budget state. If either blocks the run, it comments why and stops.
4. **Baseline.** The baseline is the latest `main` commit that has an image in ECR. Results for a baseline SHA and scenario are cached in DynamoDB, so repeated PRs against the same `main` reuse them.
5. **Probes.** Baseline (if not cached) and candidate probes launch at the same time, `R` replicas each (default 3). Each probe is one Fargate task running the app and a load runner side by side.
6. **Measure.** Each runner warms up, then sends requests at a fixed arrival rate and records latency, errors, CPU, and memory.
7. **Collect.** The CLI reads each runner's single JSON result line from CloudWatch Logs.
8. **Judge.** The decision rule compares candidate and baseline (section 6). An inconclusive result triggers one automatic re-probe with a longer run.
9. **Cost.** The cost model converts CPU and memory per request into tasks needed at the target RPS and a monthly delta (section 7).
10. **Report.** A sticky PR comment, a Check Run, an HTML report in S3, and the run record in DynamoDB.

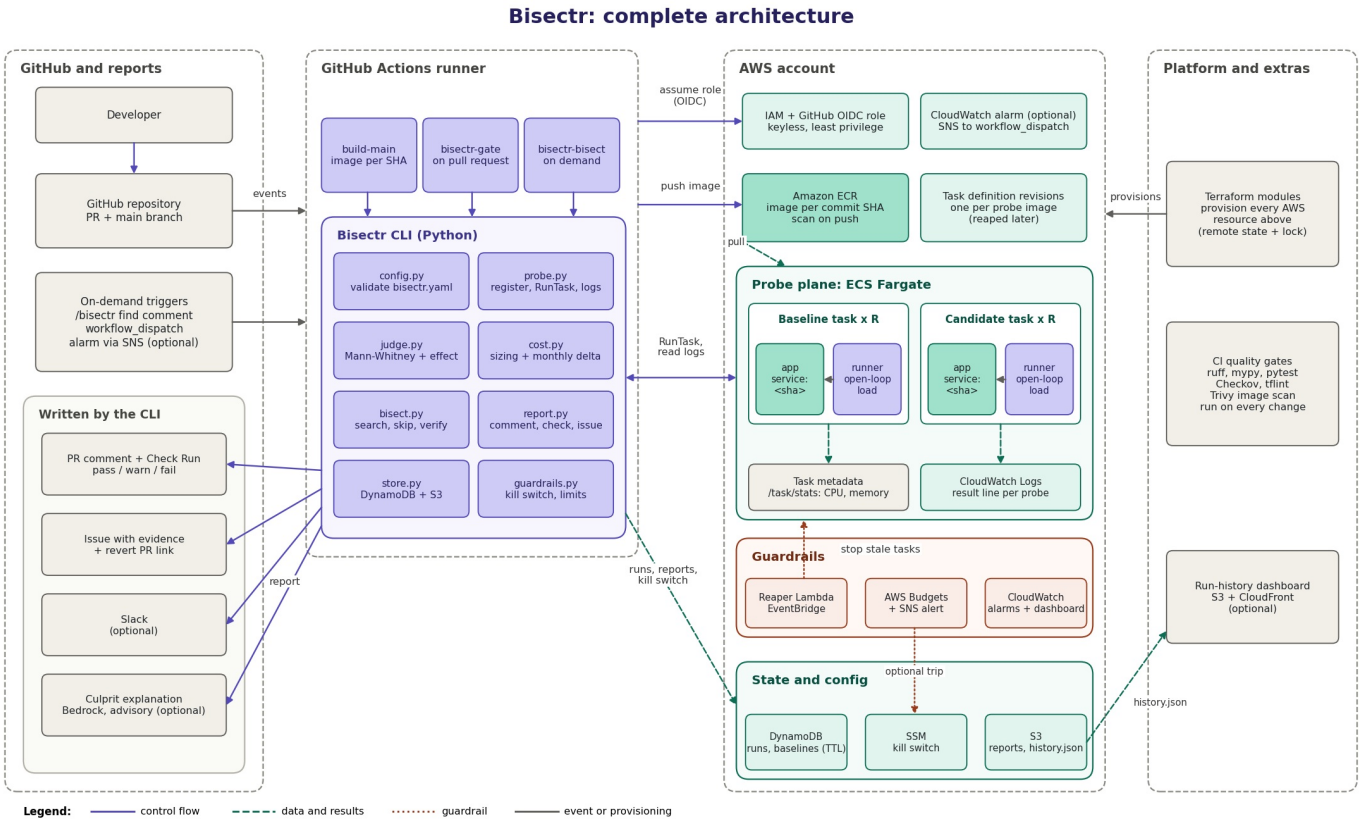
3.2 Bisect workflow



1. **Range.** The GitHub compare API lists commits between the good and bad SHAs. Commits without an ECR image are dropped, which is equivalent to `git bisect skip`.
2. **Sanity check.** The good commit must measure good and the bad commit must measure bad. If not, the run aborts with "regression not reproducible under this scenario", which is a correct answer and not a failure.
3. **Search.** Each round picks `k` midpoints (default 3), probes them in parallel, judges each against the baseline, and narrows the window. About 20 commits resolve in roughly 3 rounds.
4. **Verify.** The suspected culprit is re-measured with double the samples, and its parent is re-measured as well. The culprit must be bad and the parent good.
5. **Report.** On success, Bisectr opens an issue with the evidence and a link to a prepared revert pull request. If verification fails or skips make the window unresolvable, it reports a small suspect window or "flaky". It never reports a confident wrong commit.

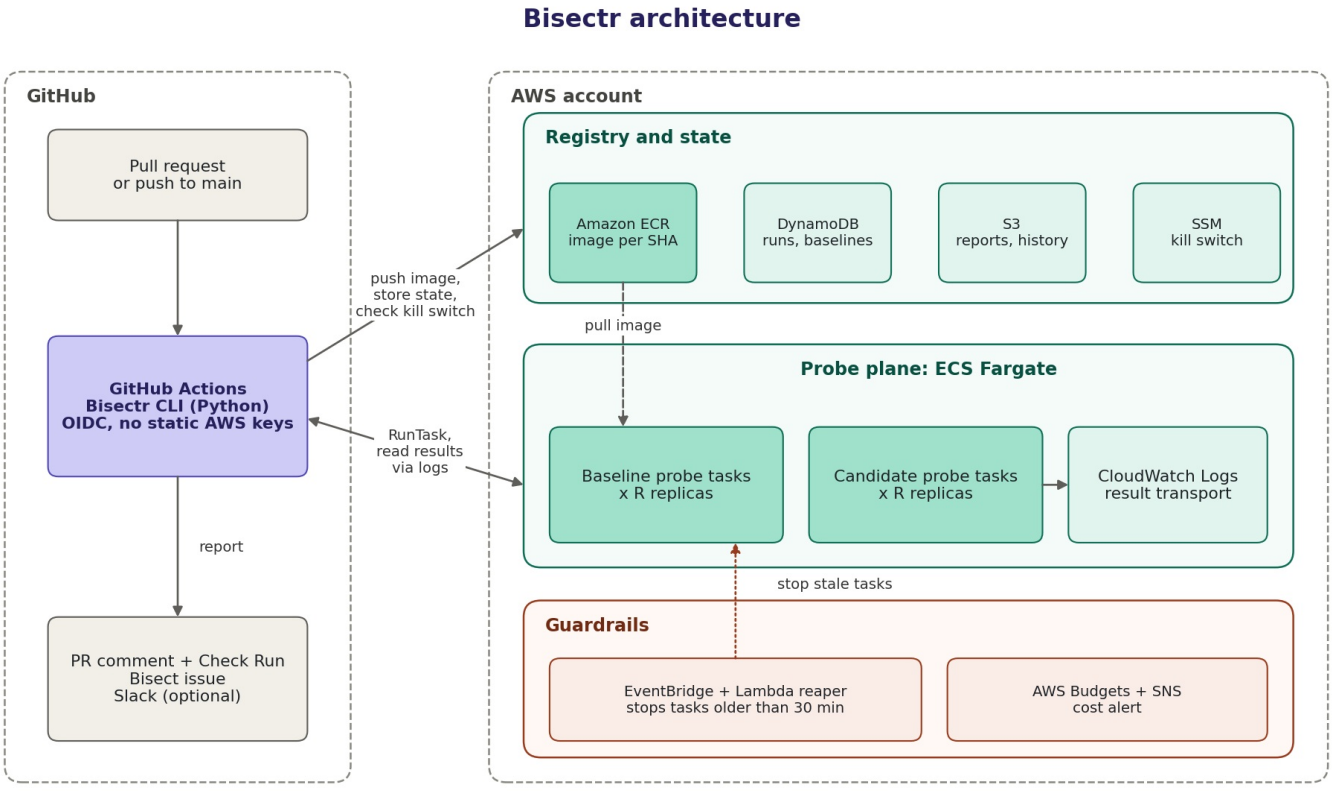
4. Architecture

Complete system view



Purple arrows are control flow, dashed teal arrows are data and results, and dotted red arrows are guardrails. A scalable SVG version is in `docs/images/architecture-complete.svg`.

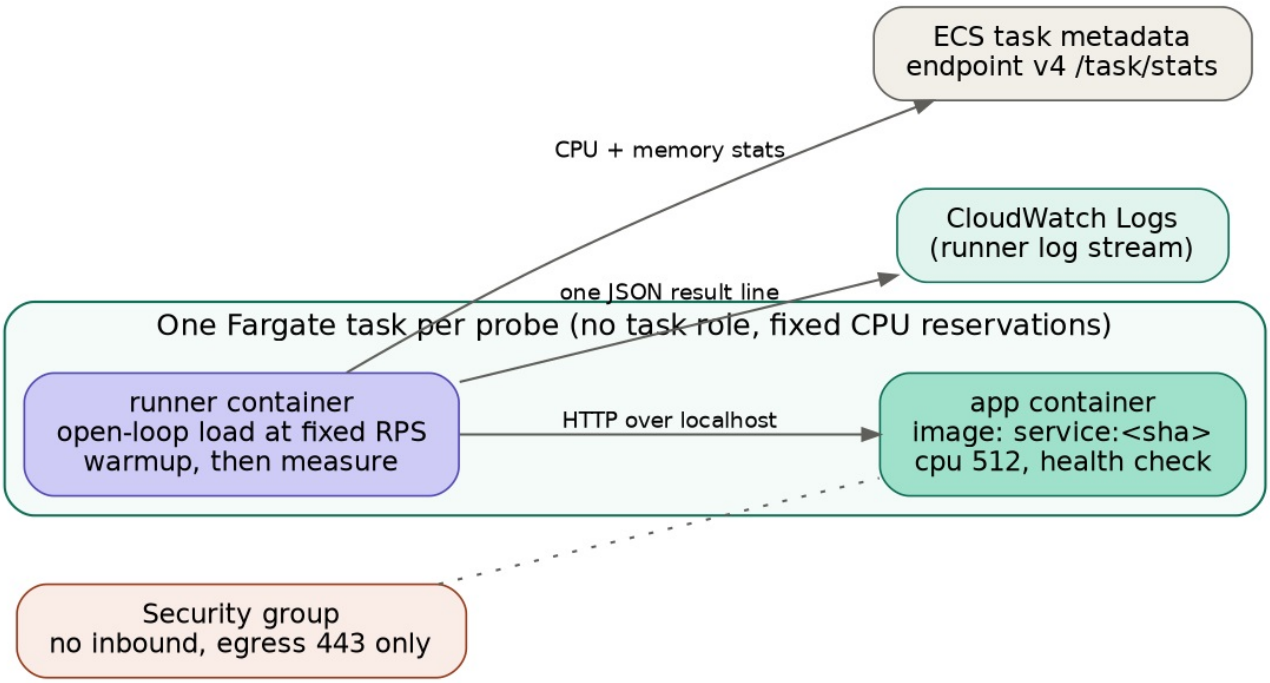
Simplified view



Component	Role
GitHub Actions + Bisectr CLI	Orchestrates everything. Authenticates to AWS with OIDC, so there are no static keys
Amazon ECR	Stores one image per commit SHA, scan-on-push enabled, lifecycle policy keeps the most recent tags
ECS on Fargate (probe plane)	Runs short-lived probe tasks for the baseline and the candidate
CloudWatch Logs	Carries each runner's result line back to the CLI
DynamoDB	Run records and cached baseline results, with TTL
S3	HTML reports and a static `history.json` for the optional dashboard
SSM Parameter Store	Kill switch (`/bisectr/enabled`)
EventBridge + Lambda reaper	Stops probe tasks older than 30 minutes and deregisters stale task definition revisions
AWS Budgets + SNS	Cost alert for the whole stack

Why no backend service? The CLI runs inside the GitHub Actions job that already has repository context and credentials. State lives in DynamoDB and S3. This removes a server to secure, patch, and pay for, and makes Bisectr easier for a real team to adopt.

5. The probe task



Each probe is one Fargate task with two containers sharing `localhost`:

- **app:** the service image tagged with the commit SHA, with a health check and a fixed CPU reservation
- **runner:** the load generator. It waits until the app is healthy, runs a discarded warmup, then measures. Its exit ends the task

Why this design

- **No network noise.** The runner talks to the app over `localhost`, not across a VPC.
- **No inbound exposure.** The security group has zero inbound rules and allows outbound traffic on port 443 only (needed to pull images and ship logs).
- **Identical conditions.** Both containers have fixed CPU reservations in every probe, so baseline and candidate are compared under the same setup.
- **Native waiting.** The CLI waits for the task to stop, then reads results.

Result transport and trust. The pull request code runs inside the app container, which must be treated as untrusted. The probe task therefore has **no task role** at all. The runner prints one JSON line (latency samples, error count, CPU, memory) to its own log stream, and the CLI reads that stream. The app container cannot write to the runner's log stream and has no AWS

credentials to forge results in DynamoDB.

CPU and memory measurement. The runner reads the ECS task metadata endpoint (v4, `/task/stats`) at the start and end of the measurement window. CPU-seconds per request is the app container's CPU delta divided by requests completed. Memory is the peak across samples. Confirm the exact fields in a short Day 1 spike before relying on them.

Task definition revisions. `RunTask` cannot override a container image, so the CLI registers a task definition revision with the candidate image for each probe. The reaper deregisters old revisions.

6. Measurement and decision rule

6.1 Open-loop load

A "send as fast as you can" tester hides slow responses because slow responses reduce the request rate. The runner sends at a fixed arrival rate and measures latency from the scheduled send time, so a stall shows up as latency instead of disappearing.

6.2 Simultaneous A/B with replicas

Baseline and candidate probes launch at the same moment, so time-of-day and noisy-neighbor drift affect both. Each side runs `R` replica tasks (default 3) on different hosts. This captures host-to-host variance that a single task would hide.

6.3 Decision rule

A candidate is `bad` only if all of these hold:

1. Mann-Whitney U test on pooled latency samples shows it is slower (`p < alpha`, default 0.01)
2. The p95 ratio is at least `min_ratio` (default 1.25), because with many samples tiny differences become "significant"
3. At least `min_agree` replicas (default 2 of 3) individually exceed the baseline median p95 by the same ratio

Error-rate increases above `error_abs` (default 2 percentage points) are `bad` on their own. Clearly small differences are `good`. Anything in between is `inconclusive` and is re-probed once with a longer run.

```
        def judge(base_reps, cand_reps, cfg):
            base = np.concatenate([r.latencies for r in base_reps])
            cand = np.concatenate([r.latencies for r in cand_reps])
            ratio = np.percentile(cand, 95) / max(np.percentile(base, 95), 1e-6)
            _, p = mannwhitneyu(cand, base, alternative="greater")
            base_med = np.median([np.percentile(r.latencies, 95) for r in base_reps])
            agree = sum(np.percentile(r.latencies, 95) / base_med >= cfg.min_ratio for r in cand_reps)
            if mean_error(cand_reps) - mean_error(base_reps) > cfg.error_abs:
                return "bad"
            if p < cfg.alpha and ratio >= cfg.min_ratio and agree >= cfg.min_agree:
                return "bad"
            if ratio < 1 + (cfg.min_ratio - 1) / 2:
                return "good"
            return "inconclusive"
```

6.4 Cost regression gate

Separately, a candidate is a **cost regression** if its median CPU-seconds per request is at least `cost_min_ratio` (default 1.15) above baseline across replicas, with the same replica agreement rule. Policy is configurable: `warn` (default) or `fail`.

6.5 Bisect search

```
def bisect(n_commits, oracle, max_probes=24):
    """oracle(i) -> 'good' | 'bad' | 'skip'. Index 0 is known good, the last is known bad."""
    lo, hi, skipped, probes = 0, n_commits - 1, set(), 0
    while hi - lo > 1 and probes < max_probes:
        mid = pick_mid(lo, hi, skipped)           # nearest non-skipped commit to the middle
        if mid is None:
            return Result("ambiguous", window=(lo, hi))
        probes += 1
        v = oracle(mid)
        if v == "skip":   skipped.add(mid)
        elif v == "bad":  hi = mid
        else:             lo = mid
    if hi - lo > 1:
        return Result("ambiguous", window=(lo, hi))
    if oracle(hi, confirm=True) == "bad" and oracle(lo, confirm=True) == "good":
        return Result("found", culprit=hi)
    return Result("flaky", window=(lo, hi))
```

7. Cost model

Fargate bills for allocated resources, not used ones, so Bisectr does not claim to predict your bill. It answers a sizing question: how many tasks would this service need at your traffic?

```
tasks_needed = ceil(target_rps * cpu_seconds_per_request / (vcpu_per_task * target_utilization))
task_hourly  = vcpu_per_task * price.vcpu_hour + gb_per_task * price.gb_hour
monthly      = tasks_needed * task_hourly * 730
delta        = monthly(candidate) - monthly(baseline)

# memory is a fit check, not a scaling factor: every task holds the app in memory
needs_bigger_task = peak_memory_mb > memory_mb_per_task * 0.8
```

- `target_rps`, `target_utilization` (default 0.6), and prices are config values. Set prices from the AWS pricing page for your region.
- The PR comment labels the number a **sizing estimate**, not a billing report.
- The model is most useful as a relative signal: "this PR needs about 23% more CPU per request".

8. Tech stack

Layer	Technology	Purpose
Language	Python 3.12	CLI, engine, runner, reaper
CLI	Typer	`gate`, `bisect`, `eval`, `report` commands
Config and validation	Pydantic, PyYAML	Validates `bisectr.yaml`
Load generation	`asyncio` + `httpx`	Open-loop runner
Statistics	NumPy, SciPy	Mann-Whitney U, percentiles, effect size
AWS SDK	boto3	ECS, ECR, Logs, DynamoDB, S3, SSM
Compute	Amazon ECS on Fargate	Probe tasks (app + runner sidecar)
Registry	Amazon ECR	One image per commit SHA, scan on push
Results transport	Amazon CloudWatch Logs	Runner result line per probe
State	Amazon DynamoDB	Runs and cached baselines, TTL
Reports	Amazon S3, Jinja2, Matplotlib	HTML report with latency distributions
Config store	AWS Systems Manager Parameter Store	Kill switch
Scheduling and cleanup	Amazon EventBridge, AWS Lambda	Reaper
Cost guard	AWS Budgets, Amazon SNS	Spend alert
Identity	IAM, GitHub OIDC	Keyless auth, least privilege

CI and delivery	GitHub Actions	Runs the gate and bisect, builds images
Source integration	GitHub REST API	Compare API, PR comments, Check Runs, issues
Infrastructure as code	Terraform (AWS provider 5.x), S3 remote state with locking	Modular infrastructure
Containers	Docker	Service images and runner image
Testing	pytest, Hypothesis, moto	Unit, property, and AWS-mock tests
Code quality	ruff, mypy	Lint and types
Security scanning	Checkov, tflint, Trivy	IaC and image scanning in CI
Optional	Amazon Bedrock	Plain-language explanation of the culprit diff (advisory only)
Optional	Slack incoming webhook	Notifications
Optional	S3 + CloudFront	Static run-history dashboard
Docs	Graphviz, Matplotlib	Diagram sources in `docs/diagrams`

9. Repository layout

```
bisectr/
  src/bisectr/
    cli.py              # Typer commands
    config.py           # Pydantic models for bisectr.yaml
    probe.py            # task definition registration, RunTask, log collection
    judge.py            # decision rule
    cost.py             # sizing and cost model
    bisect.py           # search, skip, verify
    report.py           # PR comment, Check Run, HTML report, issue
    store.py            # DynamoDB and S3
    guardrails.py       # kill switch, budget, probe limits
  runner/               # sidecar image: open-loop load + stats + result line
  action/               # GitHub Action definition
  infra/
    modules/            # network, ecr, ecs_probe, state, github_oidc, reaper, budget, web
    envs/dev/           # wiring and remote state
  lambdas/reaper/
  eval/                 # scenarios, runner, report, real_world.md
  demo-app/             # FastAPI service with injectable regressions
  tests/
  docs/
    images/             # architecture and workflow diagrams
    diagrams/           # diagram sources
    adr/                # architecture decision records
    threat-model.md
    runbook.md
  bisectr.yaml.example
  README.md
```

10. Getting started

Prerequisites

- An AWS account with permission to create ECS, ECR, IAM, DynamoDB, S3, SSM, and EventBridge resources
- A GitHub repository for the service you want to gate
- Docker, Terraform 1.5 or newer, Python 3.12

Step 1: Provision the AWS side

```
cd infra/envs/dev
terraform init
terraform apply -var="github_repo=OWNER/REPO" -var="region=ap-south-1"
terraform output        # role ARN, cluster, ECR repos, subnets, table names
```

Check your Fargate vCPU quota first. Each gate run launches `2 x R` probe tasks at once, and new accounts often have a low default quota. Request an increase or lower `replicas`.

Step 2: Add the config to your service repository

```
# bisectr.yaml
service: my-service
image_repo: <account>.dkr.ecr.ap-south-1.amazonaws.com/my-service
port: 8000
health_path: /health
task: {cpu: 1024, memory: 2048, app_cpu: 512}
scenario:
  rps: 20
  seconds: 30
  warmup_seconds: 5
  endpoints:
    - {path: /users, weight: 3}
    - {path: /orders, weight: 2}
    - {path: "/search?q=a", weight: 1}
thresholds: {alpha: 0.01, min_ratio: 1.25, min_agree: 2, error_abs: 0.02, cost_min_ratio: 1.15}
cost: {target_rps: 500, target_utilization: 0.6, on_cost_regression: warn}
pricing: {vcpu_hour: <from AWS pricing>, gb_hour: <from AWS pricing>}
replicas: 3
max_probes: 24
```

Step 3: Build an image per commit on `main`

```
# .github/workflows/build-main.yml
on: {push: {branches: [main]}}
permissions: {id-token: write, contents: read}
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: aws-actions/configure-aws-credentials@v4
        with: {role-to-assume: "${{ secrets.BISECTR_ROLE_ARN }}", aws-region: ap-south-1}
      - uses: aws-actions/amazon-ecr-login@v2
        id: ecr
      - run: |
          IMG=${{ steps.ecr.outputs.registry }}/my-service
          docker build -t $IMG:${{ github.sha }} .
          docker push $IMG:${{ github.sha }}
```

Bisect mode relies on every `main` commit having an image. For existing history, run the backfill script once.

Step 4: Add the gate to pull requests

```yaml
# .github/workflows/bisectr-gate.yml
on: pull_request
permissions: {id-token: write, contents: read, pull-requests: write, checks: write}
concurrency: {group: "bisectr-${{ github.ref }}", cancel-in-progress: true}
jobs:
  gate:
    if: github.event.pull_request.head.repo.full_name == github.repository   # skip forks
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: aws-actions/configure-aws-credentials@v4
        with: {role-to-assume: "${{ secrets.BISECTR_ROLE_ARN }}", aws-region: ap-south-1}
      - uses: aws-actions/amazon-ecr-login@v2
        id: ecr
      - run: |
          IMG=${{ steps.ecr.outputs.registry }}/my-service
          docker build -t $IMG:${{ github.event.pull_request.head.sha }} .
          docker push $IMG:${{ github.event.pull_request.head.sha }}
      - uses: OWNER/bisectr-action@v1        # your published action
        with:
          mode: gate
          config: bisectr.yaml
          candidate: ${{ github.event.pull_request.head.sha }}
```

Step 5: Run a bisect

Comment `/bisectr find <good_sha>..<bad_sha>` on a pull request or issue, or run the workflow manually:

```
bisectr bisect --config bisectr.yaml --good <good_sha> --bad <bad_sha>
```

Local development

```
pip install -e ".[dev]"
pytest                                  # unit and property tests
bisectr gate --config bisectr.yaml --candidate <sha> --baseline <sha> --dry-run
```

11. Configuration reference

Key	Default	Meaning
`scenario.rps`	20	Fixed arrival rate during measurement
`scenario.seconds`	30	Measurement window
`scenario.warmup_seconds`	5	Discarded warmup
`thresholds.alpha`	0.01	Mann-Whitney significance level
`thresholds.min_ratio`	1.25	Minimum p95 ratio to count as a regression
`thresholds.min_agree`	2	Replicas that must individually agree
`thresholds.error_abs`	0.02	Absolute error-rate increase that fails the run
`thresholds.cost_min_ratio`	1.15	CPU-per-request ratio for a cost regression
`cost.on_cost_regression`	`warn`	`warn` or `fail`
`cost.target_rps`	none	Traffic used in the sizing estimate
`cost.target_utilization`	0.6	Headroom assumed when sizing
`replicas`	3	Probe tasks per side
`max_probes`	24	Hard cap per bisect run

12. Security

Pull request code is untrusted, and this system runs it. The design assumes that.

Risk	Control
PR code reads or forges AWS data	Probe tasks have no task role. Results travel through the runner's own log stream
Network exposure	Security group has zero inbound rules and outbound 443 only
Static AWS keys in CI	GitHub OIDC with a trust policy scoped to the repository
Over-broad CI permissions	Resource-scoped IAM (below)
Forked pull requests	Gate runs only for same-repository PRs by default, because forks get no OIDC token
Abuse and runaway cost	Concurrency group per PR, `max_probes`, task timeouts, reaper, budget alert, kill switch
Vulnerable images	ECR scan on push, Trivy in CI
Secrets	GitHub token and Slack webhook in GitHub secrets or Secrets Manager, never in config files
LLM prompt injection through diffs	Bedrock output is advisory text only and never decides the culprit

Least-privilege examples for the CI role:

```
        {"Effect": "Allow", "Action": "ecs:RunTask",
         "Resource": "arn:aws:ecs:REGION:ACCOUNT:task-definition/bisectr-probe:*",
         "Condition": {"ArnEquals": {"ecs:cluster": "arn:aws:ecs:REGION:ACCOUNT:cluster/bisectr"}}}
```

```
        {"Effect": "Allow", "Action": "iam:PassRole",
         "Resource": "arn:aws:iam::ACCOUNT:role/bisectr-exec",
         "Condition": {"StringEquals": {"iam:PassedToService": "ecs-tasks.amazonaws.com"}}}
```

Also grant only: `ecs:RegisterTaskDefinition`, `ecs:DescribeTasks`, `ecs:StopTask` on the cluster; ECR push on the named repositories; `logs:GetLogEvents` on the probe log group; DynamoDB access on the two tables; `s3:PutObject` on the report prefix; `ssm:GetParameter` on the kill-switch parameter.

A threat model with the top five threats and mitigations lives in `docs/threat-model.md`.

13. Observability and cost control

- **Kill switch:** set SSM parameter `/bisectr/enabled` to `false` and all gate and bisect runs stop immediately.
- **Concurrency and limits:** per-PR concurrency group, `replicas`, `max_probes`, and a task timeout bound both spend and quota use.
- **Reaper:** an EventBridge-scheduled Lambda stops probe tasks older than 30 minutes and deregisters old task definition revisions.
- **Budget:** AWS Budgets alerts at 50% and 90% of a monthly cap through SNS.
- **Logs and metrics:** structured JSON logs from the CLI, a CloudWatch dashboard for task counts and run durations, and alarms on Lambda errors.
- **Run history:** every run is stored with its inputs, per-replica results, verdict, and cost estimate. An optional static dashboard reads `history.json` from S3.

A typical gate run launches `2 x R` short-lived tasks (6 with the defaults). Estimate cost per run with: `tasks x task_minutes x (vCPU price per minute x vCPUs + GB price per minute x GB)`, using your measured task durations.

14. Evaluation

The credibility of a performance gate comes from measured behavior, so the repository includes an evaluation harness.

`eval/scenarios.yaml` injects regressions into a demo service by patching one commit in a generated history:

Scenario	Injection	Signal
`n_plus_one`	Loop of 50 small queries replaces one	p95 latency
`slow_dependency`	Added delay on one endpoint	p95 latency
`cpu_hot_loop`	Pure CPU work per request	CPU per request, latency
`error_rate`	HTTP 500 on 10% of requests	Error rate
`payload_bloat`	Response size grows 20x	Latency, CPU

`control_noise_only`	No regression, harmless commits only	False-positive rate

```
bisectr eval --scenarios eval/scenarios.yaml --trials 20 --out eval/results.csv
python eval/report.py eval/results.csv
```

Results to publish (fill from your runs):

Metric	Result
Gate: regressions detected (excluding control)	`TODO(measure)`
Gate: false positives on noise-only control	`TODO(measure)`
Gate: cost regressions detected (`cpu_hot_loop`, `payload_bloat`)	`TODO(measure)`
Bisect: exact culprit found	`TODO(measure)`
Bisect: wrong culprit reported	`TODO(measure)`
Bisect: ambiguous window reported	`TODO(measure)`
Median wall time per gate run	`TODO(measure)`
Median probes per bisect	`TODO(measure)`
Median AWS cost per gate run	`TODO(measure)`

The metric that matters most is the **wrong-culprit and false-positive rate**. An "ambiguous" answer is safe, while a confident wrong commit or a spurious failing check destroys trust in the tool.

Real-world validation. Run Bisectr on at least one real open-source service, such as a fork of a project with a documented performance regression and a fix commit. Record the outcome in `eval/real_world.md`, including cases where it failed or was ambiguous and why.

15. Testing and CI

- **Unit and property tests:** the search and judge logic is pure, so Hypothesis checks that the search always finds the first bad commit for any monotonic history and that skips never produce a wrong answer.
- **Noise simulation:** a noisy oracle with a configurable flip rate measures accuracy and the wrong-culprit rate.
- **AWS mocks:** `moto` covers DynamoDB, SSM, ECS API calls, and log reading in the CLI.
- **CI pipeline:** `ruff` and `mypy`; `pytest` with coverage; `terraform fmt -check`, `validate`, and `tflint`; `checkov` on `infra/`; image build with `trivy` scan; Terraform plan on pull requests and apply on `main` through OIDC with a protected environment.

16. Roadmap

Four phases, about four weeks. Adjust phase lengths to the time you have, but keep the order.

Phase	Days	Goal	Done when
1. Core gate	1-8	Sidecar probe, replicated runs, judge, baseline caching, PR comment	A PR on a demo repo gets a correct pass or fail comment
2. Rigor and bisect	9-15	Cost model, bisect mode with verification, property tests, evaluation harness	`eval/results.csv` holds real numbers
3. Real-world proof	16-22	Run on one or two real repositories, tune thresholds, write an honest report	A published case study with real PR comments
4. Packaging and finish	23-28	Reusable GitHub Action, all Terraform in modules, hardening, run-history dashboard, ADRs, threat model, demo video	A fresh clone deploys from this README

Status checklist:

- [ ] Day 1 spike: confirm `/task/stats` fields and CPU-per-request measurement
- [ ] Demo app with injectable regressions
- [ ] Runner image and probe task definition
- [ ] Judge and cost model with tests
- [ ] Gate mode end to end
- [ ] Bisect mode with verification
- [ ] Evaluation harness with results

- [ ] Real-world validation case
- [ ] GitHub Action packaging
- [ ] Docs: ADRs, threat model, runbook

If you fall behind, cut in this order: the dashboard, Bedrock explanation, revert PR link, then bisect mode. Do not cut the replicated statistical judge, the verification logic, the no-task-role probe design, or the evaluation harness with real numbers.

17. Related work

Bisectr builds on ideas that already exist and applies them to a different setting.

- **Chromium Pinpoint** bisects performance regressions over ranges of changes on Google's lab infrastructure and files bugs automatically for the culprit. Bisectr adapts the approach to cloud-deployed container services.
- **airspeed velocity (asv)** benchmarks Python projects across commits and can bisect a regression. It targets libraries and benchmark suites rather than HTTP services running on cloud infrastructure.
- **CI load-test gates** are a common pattern: run a load test on the main branch, compare each pull request, and fail on regression. Bisectr adds replicated simultaneous A/B runs, a statistical decision rule with an effect-size gate, measured CPU and memory per request, and a bisect fallback.
- **AWS Cost Anomaly Detection investigations** explain cost anomalies after the fact by correlating billing data with CloudTrail activity. Bisectr works before the bill: it measures resource use per request on a pull request.
- **Infrastructure cost estimators** price resources from infrastructure code. They cannot see how efficient the application code is, which is the gap the measured CPU-per-request signal fills.

18. Limitations

- Assumes the regression is reproducible under a synthetic scenario. Regressions that only appear with production data volumes or traffic patterns are out of scope.
- The bisect search assumes the regression is roughly monotonic across the range. Reverted or intermittent regressions are reported as flaky or ambiguous.
- The runner shares CPU with the app inside one task, so absolute numbers do not equal production latency. Results are valid as relative comparisons because every probe runs under identical reservations.
- The cost output is a sizing model, not a billing report, and depends on your traffic assumption and pricing config.
- Stateful dependencies (databases, queues) must be mocked or provisioned per probe. Services with heavy external dependencies need extra setup.
- Initial scope is HTTP services in containers, hosted on Fargate, with GitHub as the source host.

19. License

MIT